

End-to-End VM Connectivity Across Subnets Using `sshuttle`, WireGuard, and Static Routing

This network slice consists of three Virtual Machines (VMs) connected across two subnets, forming a routed topology:

- **Node1 - Node2 LAN:** `192.168.1.0/24`
 - **Node2 - Node3 WAN:** `192.168.2.0/24`
-

Routing Configuration Options

1. `sshuttle` Tunneling (Automated TCP Forwarding over SSH)

As a simpler alternative to static routing, this notebook demonstrates `sshuttle`, a tool that transparently forwards TCP traffic over SSH.

- Run on **Node1**, it tunnels traffic via SSH to **Node2**
 - All packets to `192.168.2.0/24` or `192.168.3.0/24` are automatically forwarded
 - No static routes or IP forwarding setup needed on Node1
 - Ideal for fast setup, temporary debugging, or tunneling across firewalled segments
-

2. WireGuard Tunneling (Encrypted L3 Overlay)

This notebook also demonstrates [WireGuard](#), a lightweight, modern VPN protocol for creating encrypted point-to-point tunnels.

- **Node1**, **Node2**, and **Node3** form a WireGuard overlay using interface `wg0`
- Peers are connected via internal tunnel IPs in the `10.0.0.0/24` subnet
- WireGuard routes traffic securely between `192.168.1.0/24` and `192.168.2.0/24` through Node2
- Requires enabling IP forwarding and configuring `iptables` for routing between underlay and overlay

Benefits:

- Full encryption of all traffic (TCP, UDP, ICMP)
- Minimal configuration and strong performance
- Customizable per-peer routing via `AllowedIPs`

- Reliable over NAT/firewalled environments using `PersistentKeepalive`

This option is ideal for **secure overlay networks**, especially in research, educational, or partially trusted environments like **FABRIC**.

3. Static Routing (Manual Configuration)

- **Node2** manually routes traffic between `192.168.1.0/24` and `192.168.2.0/24`
- Routes are set up using `ip route` and `sysctl` to enable forwarding
- **Node3** can be similarly configured if extending beyond two hops
- No encryption, but provides transparency and control over routing behavior

This method is useful for understanding traditional IP routing principles and debugging layered topologies in testbeds.

Import the FABlib Library

```
In [85]: # Imports
from ipaddress import ip_address, IPv4Address, IPv6Address, IPv4Network, IPv6Network
import ipaddress
from fabrictestbed_extensions.fablib.fablib import FablibManager

# Initialize Fablib with your token + project
fablib = FablibManager(
    token_location="id_token.json", # adjust path if not in same folder
    project_name="Computer_Networks" # or project_id="a70de2f5-9e12-4b6b-b412-0ae1"
)

# Show current configuration
fablib.show_config()

# Verify and configure environment (fixes bastion keys if missing/expired)
fablib.verify_and_configure()
```

User: vbent01@jaguar.tamu.edu bastion key is valid!

Configuration is valid

FABlib Config

Version	1.9.3
Project Name	Computer_Networks
Token File	id_token.json
Log Level	INFO
Log File	/tmp/fablib/fablib.log
Data directory	/tmp/fablib
Orchestrator	orchestrator.fabric-testbed.net
Credential Manager	cm.fabric-testbed.net
Core API	uis.fabric-testbed.net
Artifact Manager	artifacts.fabric-testbed.net
Bastion Host	bastion.fabric-testbed.net
Bastion Private Key File	/home/fabric/work/fabric_config/fabric_bastion_key
Slice Public Key File	/home/fabric/work/fabric_config/slice_key.pub
Slice Private Key File	/home/fabric/work/fabric_config/slice_key
Sites to avoid	
SSH Command Line	ssh -i {{ _self_private_ssh_key_file }} -F /home/fabric/work/fabric_config/ssh_config {{ _self_username }}@{{ _self_management_ip }}
Bastion SSH Config File	/home/fabric/work/fabric_config/ssh_config
Fabric Meta Data Release Tag	main
Project ID	a70de2f5-9e12-4b6b-b412-0ae1a2c553b0
Bastion Username	vbent01_0000366457

User: vbent01@jaguar.tamu.edu bastion key is valid!
 Configuration is valid
 Please save the config!

Create the Experiment Slice

```
In [91]: # Slice and site configuration
slice_name = "MySlice-tunnels-static-routes"
```

```

site = "EDUKY"

node1_name = "Node1"
node2_name = "Node2"
node3_name = "Node3"

net1_name = "net1"
net2_name = "net2"
net3_name = "net3"

net1_subnet = "192.168.1.0/24"
net2_subnet = "192.168.2.0/24"
net3_subnet = "192.168.3.0/24"

image = "default_ubuntu_24"

```

```

In [92]: #Create Slice
slice = fablib.new_slice(name=slice_name)

# Network
net1 = slice.add_l2network(name=net1_name, subnet=IPv4Network(net1_subnet))
net2 = slice.add_l2network(name=net2_name, subnet=IPv4Network(net2_subnet))

# Node1
node1 = slice.add_node(name=node1_name, site=site, image=image)
iface1 = node1.add_component(model='NIC_Basic', name='nic1').get_interfaces()[0]
iface1.set_mode('auto')
net1.add_interface(iface1)

# Node2
node2 = slice.add_node(name=node2_name, site=site, image=image)
iface2 = node2.add_component(model='NIC_Basic', name='nic1').get_interfaces()[0]
iface2.set_mode('auto')
net1.add_interface(iface2)

iface3 = node2.add_component(model='NIC_Basic', name='nic2').get_interfaces()[0]
iface3.set_mode('auto')
net2.add_interface(iface3)

# Node3
node3 = slice.add_node(name=node3_name, site=site, image=image)
iface4 = node3.add_component(model='NIC_Basic', name='nic1').get_interfaces()[0]
iface4.set_mode('auto')
net2.add_interface(iface4)

#Submit Slice Request
slice.submit()

```

Retry: 12, Time: 290 sec

Slice

ID	3276abe9-b0e3-4b53-8956-0b419206741a
Name	MySlice-tunnels-static-routes
Lease Expiration (UTC)	2025-09-23 01:46:05 +0000
Lease Start (UTC)	2025-09-22 01:46:05 +0000
Project ID	a70de2f5-9e12-4b6b-b412-0ae1a2c553b0
State	StableOK
Email	vbent01@jaguar.tamu.edu
UserId	c6a0e6c5-ac3c-4631-9b9b-efb94dbf6da0

ID	Name	Cores	RAM	Disk	Image	Image Type	Host	Site	Us
4b6cedcd-9dfc-4cb4-95a4-bfd262846d44	Node1	2	8	10	default_ubuntu_24	qcow2	eduky-w10.fabric-testbed.net	EDUKY	ub
2b3a128d-83a7-41d7-99e6-181fc5ece352	Node2	2	8	10	default_ubuntu_24	qcow2	eduky-w10.fabric-testbed.net	EDUKY	ub
5e7ea8c8-0261-44a9-8c54-10e6792978d4	Node3	2	8	10	default_ubuntu_24	qcow2	eduky-w15.fabric-testbed.net	EDUKY	ub



Networks

ID	Name	Layer	Type	Site	Subnet	Gateway	State	Error
a116688e-6891-4786-959f-c8e1958abe7a	net1	L2	L2Bridge	EDUKY	192.168.1.0/24	None	Active	
6ead3a5b-3b14-44b8-8ae9-e93b1acb8e87	net2	L2	L2Bridge	EDUKY	192.168.2.0/24	None	Active	

Interfaces

Name	Short Name	Node	Network	Bandwidth	Mode	VLAN	MAC	Physical Device	De
Node3-nic1-p1	p1	Node3	net2	100	auto		06:66:2B:9F:0D:B8	enp7s0	en
Node2-nic1-p1	p1	Node2	net1	100	auto		0E:A0:92:D7:1C:49	enp8s0	en
Node2-nic2-p1	p1	Node2	net2	100	auto		0A:F8:61:E4:D9:29	enp7s0	en
Node1-nic1-p1	p1	Node1	net1	100	auto		16:4E:A9:5A:2E:9E	enp7s0	en



Time to print interfaces 291 seconds

Out[92]: '3276abe9-b0e3-4b53-8956-0b419206741a'

Verify Connectivity

Before proceeding with routing or tunneling setup, ensure that basic IP-layer connectivity exists between directly connected nodes:

- Check that **Node1 and Node2** are reachable from each other
- Check that **Node2 and Node3** are reachable from each other

```
In [93]: # Check ping on Site1 to Site2 (L2STS)
slice = fablib.get_slice(slice_name)

node1 = slice.get_node(name=node1_name)
node2 = slice.get_node(name=node2_name)

node2_addr = node2.get_interface(network_name=net1_name).get_ip_addr()

stdout, stderr = node1.execute(f'ping -c 5 {node2_addr}')
```

```
PING 192.168.1.2 (192.168.1.2) 56(84) bytes of data.
64 bytes from 192.168.1.2: icmp_seq=1 ttl=64 time=0.355 ms
64 bytes from 192.168.1.2: icmp_seq=2 ttl=64 time=0.130 ms
64 bytes from 192.168.1.2: icmp_seq=3 ttl=64 time=0.125 ms
64 bytes from 192.168.1.2: icmp_seq=4 ttl=64 time=0.114 ms
64 bytes from 192.168.1.2: icmp_seq=5 ttl=64 time=0.143 ms

--- 192.168.1.2 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4082ms
rtt min/avg/max/mdev = 0.114/0.173/0.355/0.091 ms
```

```
In [94]: # Check ping on Site2 to Site3 (L2STS)
slice = fablib.get_slice(slice_name)

node2 = slice.get_node(name=node2_name)
```

```
node3 = slice.get_node(name=node3_name)

node3_addr = node3.get_interface(network_name=net2_name).get_ip_addr()

stdout, stderr = node2.execute(f'ping -c 5 {node3_addr}')
```

```
PING 192.168.2.2 (192.168.2.2) 56(84) bytes of data.
64 bytes from 192.168.2.2: icmp_seq=1 ttl=64 time=0.317 ms
64 bytes from 192.168.2.2: icmp_seq=2 ttl=64 time=0.146 ms
64 bytes from 192.168.2.2: icmp_seq=3 ttl=64 time=0.123 ms
64 bytes from 192.168.2.2: icmp_seq=4 ttl=64 time=0.136 ms
64 bytes from 192.168.2.2: icmp_seq=5 ttl=64 time=0.166 ms
```

```
--- 192.168.2.2 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4110ms
rtt min/avg/max/mdev = 0.123/0.177/0.317/0.071 ms
```

Configure SSH Key-Based Access

Generate SSH key pairs for both the `ubuntu` and `root` users.

Distribute the corresponding public keys to all nodes by appending them to the appropriate `authorized_keys` files, enabling passwordless SSH access between nodes.

```
In [95]: for n in slice.get_nodes():
         n.execute('ssh-keygen -t rsa -N "" -f /home/ubuntu/.ssh/id_rsa', quiet=True)
         n.execute('sudo ssh-keygen -t rsa -N "" -f /root/.ssh/id_rsa', quiet=True)
```

```
In [96]: keys = {}
         # Step 1: Collect public keys from each node
         for n in slice.get_nodes():
             ubuntu_key, _ = n.execute("cat /home/ubuntu/.ssh/id_rsa.pub", quiet=True)
             root_key, _ = n.execute("sudo cat /root/.ssh/id_rsa.pub", quiet=True)
             keys[n.get_name()] = {
                 "ubuntu": ubuntu_key.strip(),
                 "root": root_key.strip()
             }

         # Step 2: Distribute public keys to all other nodes
         for n in slice.get_nodes():
             for other_node_name, node_keys in keys.items():
                 if other_node_name == n.get_name():
                     continue
                 n.execute(f'echo "{node_keys["ubuntu"]}" >> /home/ubuntu/.ssh/authorized_keys')
                 n.execute(f'sudo sh -c \'echo "{node_keys["root"]}" >> /root/.ssh/authorized_keys\'')
```

Option 1: `sshuttle` — Simplified Tunneling Over SSH

`sshuttle` provides a transparent, TCP-based VPN-like tunnel using only SSH. This option simplifies cross-node connectivity by automatically forwarding traffic from one node to a remote subnet without requiring static routes or manual IP forwarding setup.

In this configuration:

- `sshuttle` runs on **Node1** and creates an SSH tunnel to **Node2**
- Any TCP traffic destined for the `192.168.2.0/24` subnets is captured and forwarded through this tunnel
- No modifications are needed on **Node3** or beyond, as long as Node2 can reach them

Why Use `sshuttle`?

- You want a quick and minimal setup
- You do not have `sudo` access on all nodes to configure IP forwarding
- You're working in a firewalled or restricted environment

⚠ Note: `sshuttle` only supports TCP traffic (e.g., SSH, HTTP) — not ICMP (`ping`) or UDP-based protocols.

```
In [97]: slice = fablib.get_slice(slice_name)
node1 = slice.get_node(name=node1_name)
node2 = slice.get_node(name=node2_name)
node3 = slice.get_node(name=node3_name)
```

```
In [98]: node1_net1_addr = node1.get_interface(network_name=net1_name).get_ip_addr()
node2_net1_addr = node2.get_interface(network_name=net1_name).get_ip_addr()

node2_net2_addr = node2.get_interface(network_name=net2_name).get_ip_addr()
node3_net2_addr = node3.get_interface(network_name=net2_name).get_ip_addr()
```

```
In [99]: for n in slice.get_nodes():
stdout, stderr = n.execute("sudo apt update && sudo apt install -y sshuttle net
```

Start `sshuttle` from Node1 and Add a Dummy Route

To enable dynamic tunneling from **Node1** to the remote subnet via **Node2**, follow these steps:

1. Launch `sshuttle` on Node1

Run the following command with `sudo` to start tunneling TCP traffic for the target subnet:

```
sudo sshuttle --method=nat \
  --ssh-cmd 'ssh -o StrictHostKeyChecking=no -o
UserKnownHostsFile=/dev/null' \
  -r root@192.168.1.1 \
  192.168.2.0/24 \
  -vv
```



```
In [100... node1 = slice.get_node(node1_name)
cmd = f"sudo sshuttle --method=nat --ssh-cmd 'ssh -o StrictHostKeyChecking=no -o
print(f"Executing: {cmd}")
stdout, stderr = node1.execute(cmd)
```

```
Executing: sudo sshuttle --method=nat --ssh-cmd 'ssh -o StrictHostKeyChecking=no -
o UserKnownHostsFile=/dev/null' -r root@192.168.1.2 192.168.2.0/24 --daemon
```

2. Add a Dummy Route to Force Traffic into iptables

On some systems, the kernel won't allow packets to be emitted for unreachable networks — which prevents sshuttle from seeing them. To fix this, add a dummy route via loopback:

This route doesn't send traffic to loopback — it simply convinces the kernel to emit packets, allowing iptables to redirect them through sshuttle.

```
In [101... # Add a dummy route to force traffic to hit iptables
stdout, stderr = node1.execute(f"sudo ip route add {net2_subnet} via 127.0.0.1 dev
```

3. Test the Tunnel

Send a TCP packet (e.g., to port 22) to confirm redirection:

You should now be able to access Node3 as if directly connected.

```
In [102... stdout, stderr = node1.execute(f'nc -zv {node3_net2_addr} 22')
```

```
Connection to 192.168.2.2 22 port [tcp/ssh] succeeded!
```

```
In [103... !ip route show
```

```
default via 10.36.2.1 dev eth0
10.36.2.0/24 via 10.36.2.1 dev eth0 src 10.36.2.151
10.36.2.1 dev eth0 scope link src 10.36.2.151
```

```
In [104... slice = fablib.get_slice(name="MySlice-tunnels-static-routes")

node1 = slice.get_node("Node1")
node2 = slice.get_node("Node2")
node3 = slice.get_node("Node3")

for node in [node1, node2, node3]:
    print(f"== {node.get_name()} routing table ==")
    out, _ = node.execute("ip route show")
    print(out)
```

```

== Node1 routing table ==
10.30.0.0/19 dev enp3s0 proto kernel scope link src 10.30.6.111 metric 100
10.30.6.11 dev enp3s0 proto dhcp scope link src 10.30.6.111 metric 100
169.254.169.254 via 10.30.6.11 dev enp3s0 proto dhcp src 10.30.6.111 metric 100
192.168.1.0/24 dev enp7s0 proto kernel scope link src 192.168.1.1
192.168.2.0/24 via 127.0.0.1 dev lo
10.30.0.0/19 dev enp3s0 proto kernel scope link src 10.30.6.111 metric 100
10.30.6.11 dev enp3s0 proto dhcp scope link src 10.30.6.111 metric 100
169.254.169.254 via 10.30.6.11 dev enp3s0 proto dhcp src 10.30.6.111 metric 100
192.168.1.0/24 dev enp7s0 proto kernel scope link src 192.168.1.1
192.168.2.0/24 via 127.0.0.1 dev lo

== Node2 routing table ==
10.30.0.0/19 dev enp3s0 proto kernel scope link src 10.30.7.130 metric 100
10.30.6.11 dev enp3s0 proto dhcp scope link src 10.30.7.130 metric 100
169.254.169.254 via 10.30.6.11 dev enp3s0 proto dhcp src 10.30.7.130 metric 100
192.168.1.0/24 dev enp8s0 proto kernel scope link src 192.168.1.2
192.168.2.0/24 dev enp7s0 proto kernel scope link src 192.168.2.1
10.30.0.0/19 dev enp3s0 proto kernel scope link src 10.30.7.130 metric 100
10.30.6.11 dev enp3s0 proto dhcp scope link src 10.30.7.130 metric 100
169.254.169.254 via 10.30.6.11 dev enp3s0 proto dhcp src 10.30.7.130 metric 100
192.168.1.0/24 dev enp8s0 proto kernel scope link src 192.168.1.2
192.168.2.0/24 dev enp7s0 proto kernel scope link src 192.168.2.1

== Node3 routing table ==
10.30.0.0/19 dev enp3s0 proto kernel scope link src 10.30.6.236 metric 100
10.30.6.11 dev enp3s0 proto dhcp scope link src 10.30.6.236 metric 100
169.254.169.254 via 10.30.6.11 dev enp3s0 proto dhcp src 10.30.6.236 metric 100
192.168.2.0/24 dev enp7s0 proto kernel scope link src 192.168.2.2
10.30.0.0/19 dev enp3s0 proto kernel scope link src 10.30.6.236 metric 100
10.30.6.11 dev enp3s0 proto dhcp scope link src 10.30.6.236 metric 100
169.254.169.254 via 10.30.6.11 dev enp3s0 proto dhcp src 10.30.6.236 metric 100
192.168.2.0/24 dev enp7s0 proto kernel scope link src 192.168.2.2

```

```

In [105... # Clean sshuttle verification: show sshuttle route + connectivity test
sshuttle_subnet = "192.168.2.0/24" # subnet tunneled with sshuttle

slice = fablib.get_slice(name="MySlice-tunnels-static-routes")
node1 = slice.get_node("Node1")
node2 = slice.get_node("Node2")
node3 = slice.get_node("Node3")

# Check routing tables
for node in [node1, node2, node3]:
    print(f"== {node.get_name()} sshuttle Routes ==")
    out, _ = node.execute("ip route show")
    filtered = [line for line in out.splitlines() if sshuttle_subnet in line]
    if filtered:
        for line in filtered:
            print(line)
    else:
        print("(no sshuttle routes)")

# Connectivity test: Node1 → Node3 through sshuttle
print("\n== sshuttle Connectivity Test (Node1 → Node3 port 22) ==")

```

```
out, _ = node1.execute("nc -zv 192.168.2.2 22")
print(out if out else "(nc returned no output)")
```

```
== Node1 sshuttle Routes ==
10.30.0.0/19 dev enp3s0 proto kernel scope link src 10.30.6.111 metric 100
10.30.6.11 dev enp3s0 proto dhcp scope link src 10.30.6.111 metric 100
169.254.169.254 via 10.30.6.11 dev enp3s0 proto dhcp src 10.30.6.111 metric 100
192.168.1.0/24 dev enp7s0 proto kernel scope link src 192.168.1.1
192.168.2.0/24 via 127.0.0.1 dev lo
192.168.2.0/24 via 127.0.0.1 dev lo
== Node2 sshuttle Routes ==
10.30.0.0/19 dev enp3s0 proto kernel scope link src 10.30.7.130 metric 100
10.30.6.11 dev enp3s0 proto dhcp scope link src 10.30.7.130 metric 100
169.254.169.254 via 10.30.6.11 dev enp3s0 proto dhcp src 10.30.7.130 metric 100
192.168.1.0/24 dev enp8s0 proto kernel scope link src 192.168.1.2
192.168.2.0/24 dev enp7s0 proto kernel scope link src 192.168.2.1
192.168.2.0/24 dev enp7s0 proto kernel scope link src 192.168.2.1
== Node3 sshuttle Routes ==
10.30.0.0/19 dev enp3s0 proto kernel scope link src 10.30.6.236 metric 100
10.30.6.11 dev enp3s0 proto dhcp scope link src 10.30.6.236 metric 100
169.254.169.254 via 10.30.6.11 dev enp3s0 proto dhcp src 10.30.6.236 metric 100
192.168.2.0/24 dev enp7s0 proto kernel scope link src 192.168.2.2
192.168.2.0/24 dev enp7s0 proto kernel scope link src 192.168.2.2

== sshuttle Connectivity Test (Node1 → Node3 port 22) ==
Connection to 192.168.2.2 22 port [tcp/ssh] succeeded!
(nc returned no output)
```

Option 2: WireGuard Tunnel (Encrypted L3 Tunnel)

WireGuard is a modern, lightweight, high-performance VPN protocol suitable for setting up encrypted tunnels across routed networks like FABRIC.

This option demonstrates how to use WireGuard to tunnel traffic between **Node1** and **Node3** via **Node2**, forming an L3 encrypted overlay.

Why Use WireGuard?

- **End-to-End Encryption:** All traffic between Node1 and Node3 is encrypted.
- **Bypasses Static Routing Complexity:** Tunnel hides underlying subnet structure, reducing the need for route manipulation.
- **Protocol Flexibility:** Supports ICMP, TCP, UDP—unlike sshuttle which is TCP-only.
- **Lightweight and Fast:** Minimal performance overhead, ideal for research environments.

Topology Overview

- **Underlay Subnets:**
 - Node1 ↔ Node2 (LAN): 192.168.1.0/24
 - Node2 ↔ Node3 (WAN): 192.168.2.0/24

- **WireGuard Overlay (Tunnel IPs):**

- Node1: 10.0.0.1
- Node2: 10.0.0.2
- Node3: 10.0.0.3

Node2 acts as a WireGuard relay for Node1 ↔ Node3, and also bridges overlay-to-underlay with IP forwarding and appropriate `iptables` rules.

When to Use This Option

- When encrypted inter-node communication is required
- When you want better protocol support than sshuttle
- When route configuration is constrained or complex

In the next sections, we will:

- Install and configure WireGuard on all three nodes
- Establish persistent tunnels
- Verify encrypted ping and connectivity between Node1 and Node3

In [106...

```
slice = fablib.get_slice(slice_name)
node1 = slice.get_node(name=node1_name)
node2 = slice.get_node(name=node2_name)
node3 = slice.get_node(name=node3_name)

node1_net1_addr = node1.get_interface(network_name=net1_name).get_ip_addr()
node2_net1_addr = node2.get_interface(network_name=net1_name).get_ip_addr()

node2_net2_ifc = node2.get_interface(network_name=net2_name)
node2_net2_addr = node2_net2_ifc.get_ip_addr()
node2_net2_ifc_name = node2_net2_ifc.get_os_interface()

node3_net2_addr = node3.get_interface(network_name=net2_name).get_ip_addr()
```

Cleanup Any Leftover Configs Before Setting Up WireGuard

Before configuring WireGuard, ensure the system is free of conflicting settings that might interfere with the tunnel.

- **Remove any static routes:**

Run the following on each node as needed:

```
sudo ip route del <ROUTE>
```

- **Stop sshuttle:**

If sshuttle was started in daemon mode:

```
sudo pkill -f sshuttle
```

```
In [107... stdout, stderr = node1.execute(f"sudo ip route del {net2_subnet}")
          stdout, stderr = node3.execute(f"sudo ip route del {net1_subnet}")
```

RTNETLINK answers: No such process

```
In [108... stdout, stderr = node1.execute(f"sudo pkill -f sshuttle")
          stdout, stderr = node3.execute(f"sudo pkill -f sshuttle")
```

1. Install WireGuard on All Nodes and Generate WireGuard Key Pairs

WireGuard needs to be installed on each of the nodes (**Node1**, **Node2**, and **Node3**) participating in the tunnel.

```
In [109... keys = {}
for n in slice.get_nodes():
    stdout, stderr = n.execute("sudo apt-get update && sudo apt-get install -y wire
    stdout, stderr = n.execute("wg genkey | tee privatekey | wg pubkey > publickey"
    priv_key, stderr = n.execute("cat privatekey", quiet=True)
    pub_key, stderr = n.execute("cat publickey", quiet=True)
    keys[n.get_name()] = {
        "public": pub_key,
        "private": priv_key,
    }
```

2. Configure WireGuard Interfaces

Setup WireGuard on Node1

Create the configuration file `/etc/wireguard/wg0.conf` on **Node1**:

```
In [110... node1_wg_conf = f"""\
[Interface]
PrivateKey = {keys.get(node1.get_name(), {}).get('private')}
Address = 10.0.0.1/24
ListenPort = 51820

[Peer]
PublicKey = {keys.get(node2.get_name(), {}).get('public')}
Endpoint = {node2_net1_addr}:51820
AllowedIPs = 10.0.0.0/24, {net2_subnet}
PersistentKeepalive = 25
"""

stdout, stderr = node1.execute(f"echo '{node1_wg_conf}' | sudo tee /etc/wireguard/w

stdout, stderr = node1.execute("sudo systemctl enable wg-quick@wg0 && sudo systemct
```

Created symlink `/etc/systemd/system/multi-user.target.wants/wg-quick@wg0.service` → `/usr/lib/systemd/system/wg-quick@.service`.

Setup WireGuard on Node2

Create the configuration file `/etc/wireguard/wg0.conf` on **Node2**:

```
In [111... node2_wg_conf = f"""\n
[Interface]
PrivateKey = {keys.get(node2.get_name(), {}).get('private')}
Address = 10.0.0.2/24
ListenPort = 51820
PostUp = iptables -A FORWARD -i %i -j ACCEPT; iptables -A FORWARD -o %i -j ACCEPT;
PostDown = iptables -D FORWARD -i %i -j ACCEPT; iptables -D FORWARD -o %i -j ACCEPT

[Peer]
PublicKey = {keys.get(node1.get_name(), {}).get('public')}
Endpoint = {node1_net1_addr}:51820
AllowedIPs = 10.0.0.1/32
PersistentKeepalive = 25

[Peer]
PublicKey = {keys.get(node3.get_name(), {}).get('public')}
Endpoint = {node3_net2_addr}:51820
AllowedIPs = 10.0.0.3/32
PersistentKeepalive = 25
"""

stdout, stderr = node2.execute(f"echo '{node2_wg_conf}' | sudo tee /etc/wireguard/w
stdout, stderr = node2.execute("sudo systemctl enable wg-quick@wg0 && sudo systemct
```

Created symlink `/etc/systemd/system/multi-user.target.wants/wg-quick@wg0.service` → `/usr/lib/systemd/system/wg-quick@.service`.

Setup WireGuard on Node3

Create the configuration file `/etc/wireguard/wg0.conf` on **Node3**:

```
In [112... node3_wg_conf = f"""\n
[Interface]
PrivateKey = {keys.get(node3.get_name(), {}).get('private')}
Address = 10.0.0.3/24
ListenPort = 51820

[Peer]
PublicKey = {keys.get(node2.get_name(), {}).get('public')}
Endpoint = {node2_net2_addr}:51820
AllowedIPs = 10.0.0.0/24, {net1_subnet}
PersistentKeepalive = 25
"""

stdout, stderr = node3.execute(f"echo '{node3_wg_conf}' | sudo tee /etc/wireguard/w
stdout, stderr = node3.execute("sudo systemctl enable wg-quick@wg0 && sudo systemct
```

Created symlink `/etc/systemd/system/multi-user.target.wants/wg-quick@wg0.service` → `/usr/lib/systemd/system/wg-quick@.service`.

4. Enable IP Forwarding on Node2

To allow Node2 to route traffic between `Node1` and `Node3` over the WireGuard tunnel, enable IPv4 forwarding:

```
In [113... stdout, stderr = node2.execute("echo 1 | sudo tee /proc/sys/net/ipv4/ip_forward &&  
1  
net.ipv4.ip_forward = 1
```

5. Test Tunnel

Once the WireGuard interfaces are configured and active on all nodes, and IP forwarding is enabled on Node2, you can verify tunnel connectivity.

From Node1:

Try pingping Node3's WireGuard IP or private interface IP:

```
ping 10.0.0.3
```

From Node3:

Try pingping Node1's WireGuard IP or private interface IP:

```
ping 10.0.0.1
```

If the tunnel is working correctly, you should see successful replies. You can also inspect WireGuard status and packet counts with:

```
sudo wg show
```

This confirms end-to-end encrypted L3 connectivity through the WireGuard tunnel.

```
In [114... stdout, stderr = node1.execute(f'ping -c 5 10.0.0.3')  
stdout, stderr = node1.execute(f'ping -c 5 {node3_net2_addr}')
```

```

PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
64 bytes from 10.0.0.3: icmp_seq=1 ttl=63 time=0.621 ms
64 bytes from 10.0.0.3: icmp_seq=2 ttl=63 time=0.553 ms
64 bytes from 10.0.0.3: icmp_seq=3 ttl=63 time=0.471 ms
64 bytes from 10.0.0.3: icmp_seq=4 ttl=63 time=0.566 ms
64 bytes from 10.0.0.3: icmp_seq=5 ttl=63 time=0.423 ms

--- 10.0.0.3 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4105ms
rtt min/avg/max/mdev = 0.423/0.526/0.621/0.070 ms
PING 192.168.2.2 (192.168.2.2) 56(84) bytes of data.
64 bytes from 192.168.2.2: icmp_seq=1 ttl=63 time=0.371 ms
64 bytes from 192.168.2.2: icmp_seq=2 ttl=63 time=0.389 ms
64 bytes from 192.168.2.2: icmp_seq=3 ttl=63 time=0.403 ms
64 bytes from 192.168.2.2: icmp_seq=4 ttl=63 time=0.391 ms
64 bytes from 192.168.2.2: icmp_seq=5 ttl=63 time=0.405 ms

--- 192.168.2.2 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4123ms
rtt min/avg/max/mdev = 0.371/0.391/0.405/0.012 ms

```

```

In [115... stdout, stderr = node3.execute(f'ping -c 5 10.0.0.1')
          stdout, stderr = node3.execute(f'ping -c 5 {node1_net1_addr}')

```

```

PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data.
64 bytes from 10.0.0.1: icmp_seq=1 ttl=63 time=0.516 ms
64 bytes from 10.0.0.1: icmp_seq=2 ttl=63 time=0.563 ms
64 bytes from 10.0.0.1: icmp_seq=3 ttl=63 time=0.502 ms
64 bytes from 10.0.0.1: icmp_seq=4 ttl=63 time=0.977 ms
64 bytes from 10.0.0.1: icmp_seq=5 ttl=63 time=0.536 ms

--- 10.0.0.1 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4090ms
rtt min/avg/max/mdev = 0.502/0.618/0.977/0.180 ms
PING 192.168.1.1 (192.168.1.1) 56(84) bytes of data.

--- 192.168.1.1 ping statistics ---
5 packets transmitted, 0 received, 100% packet loss, time 4090ms

```

```

In [116... stdout, stderr = node1.execute(f'nc -zv {node3_net2_addr} 22')

```

Connection to 192.168.2.2 22 port [tcp/ssh] succeeded!

```

In [117... # Map WireGuard public keys to node names
peer_map = {
    "qfWlUh7lxxGavDlcvXR5tbhbnBXwupZ/HVBgsOrXd1w=": "Node1",
    "pFRC5BdvGf4XBfZhJHaTD6s4E6cVeQ/3zw8s2CDgah4=": "Node2",
    "tBbDxeT/iMqunGa6glkkSP/rsIzf5P0xDgh2BMyvMiQ=": "Node3"
}

def parse_transfer(output):
    summary = {}
    for line in output.strip().split("\n"):
        parts = line.split("\t")
        if len(parts) == 4:
            _, peer, rx, tx = parts

```



```

        rx, tx = int(rx), int(tx)
        if peer not in summary:
            summary[peer] = [rx, tx]
        else:
            summary[peer][0] += rx
            summary[peer][1] += tx
    return summary

for node in [node1, node2, node3]:
    print(f"== {node.get_name()} WireGuard Summary ==")
    out, _ = node.execute("sudo wg show all transfer")
    stats = parse_transfer(out)
    for peer, (rx, tx) in stats.items():
        name = peer_map.get(peer, peer) # map to node name if known
        print(f"Peer {name}: {rx/1024:.2f} KB received, {tx/1024:.2f} KB sent")

```

```

== Node1 WireGuard Summary ==
wg0      QJzqhjw2A1kpgheg906IljEZJI+TiYpxrBLnNJK81Q=      2548      3200
Peer QJzqhjw2A1kpgheg906IljEZJI+TiYpxrBLnNJK81Q=: 2.49 KB received, 3.12 KB sent
== Node2 WireGuard Summary ==
wg0      cUBuERagidCt7cvLFSa+lZiXs6DXEKB1jPjtSRyl1Bc=      2412      2548
wg0      d5TkAg5lH5Xa8TzIqU2MTP4jMWPWhvddqXfhW9cqdmA=      2164      1584
Peer cUBuERagidCt7cvLFSa+lZiXs6DXEKB1jPjtSRyl1Bc=: 2.36 KB received, 2.49 KB sent
Peer d5TkAg5lH5Xa8TzIqU2MTP4jMWPWhvddqXfhW9cqdmA=: 2.11 KB received, 1.55 KB sent
== Node3 WireGuard Summary ==
wg0      QJzqhjw2A1kpgheg906IljEZJI+TiYpxrBLnNJK81Q=      1436      2164
Peer QJzqhjw2A1kpgheg906IljEZJI+TiYpxrBLnNJK81Q=: 1.40 KB received, 2.11 KB sent

```

Option 3: Static Routes and IP Forwarding

This approach demonstrates how to manually configure static routes and enable IP forwarding to achieve end-to-end connectivity across subnets.

In this setup:

- **Node2** acts as a router between **Node1** and **Node3**, bridging **192.168.1.0/24** and **192.168.2.0/24**
- Each node's routing table is explicitly updated using **ip route** commands
- IP forwarding is enabled on intermediary nodes to allow packets to be relayed between interfaces

Why Use Static Routes?

- You want full control and visibility into packet routing
- You're building realistic network experiments
- You need to support all protocols (TCP, UDP, ICMP, etc.)
- Unlike sshuttle, this approach supports all IP traffic, not just TCP.

```

In [118... slice = fablib.get_slice(slice_name)
node1 = slice.get_node(name=node1_name)
node2 = slice.get_node(name=node2_name)

```

```
node3 = slice.get_node(name=node3_name)

node1_net1_addr = node1.get_interface(network_name=net1_name).get_ip_addr()
node2_net1_addr = node2.get_interface(network_name=net1_name).get_ip_addr()

node2_net2_addr = node2.get_interface(network_name=net2_name).get_ip_addr()
node3_net2_addr = node3.get_interface(network_name=net2_name).get_ip_addr()
```

1. Cleanup Any Leftover Configs Before Setting Up Static Routes

Before configuring WireGuard, ensure the system is free of conflicting settings that might interfere with the tunnel.

- **Remove any static routes:**

Run the following on each node as needed:

```
sudo ip route del <ROUTE>
```

- **Stop sshuttle:**

If sshuttle was started in daemon mode:

```
sudo pkill -f sshuttle
```

- **Stop wireguard:**

If sshuttle was started in daemon mode:

```
sudo pkill -f sshuttle
```

```
In [119... stdout, stderr = node1.execute(f"sudo ip route del {net2_subnet} via 127.0.0.1 dev
RTNETLINK answers: No such process
```

2. Enable IP Forwarding

On **Node2**, enable IP forwarding:

```
sudo sysctl -w net.ipv4.ip_forward=1
```

```
In [120... stdout, stderr = node2.execute("echo 1 | sudo tee /proc/sys/net/ipv4/ip_forward &&
1
net.ipv4.ip_forward = 1
```

3. Set Up Static Routes

- Add a static route on Node1 to reach Node3 via Node2
- Add a static route on Node3 to reach Node1 via Node2

```
In [121... node1_net1_addr = node1.get_interface(network_name=net1_name).get_ip_addr()
node2_net1_addr = node2.get_interface(network_name=net1_name).get_ip_addr()

node2_net2_addr = node2.get_interface(network_name=net2_name).get_ip_addr()
node3_net2_addr = node3.get_interface(network_name=net2_name).get_ip_addr()

print("Setup routes on Node1")
stdout, stderr = node1.execute(f"sudo ip route add {net2_subnet} via {node2_net1_ad
stdout, stderr = node1.execute(f"sudo ip route list")

print()
print("Setup routes on Node3")
stdout, stderr = node3.execute(f"sudo ip route add {net1_subnet} via {node2_net2_ad
stdout, stderr = node3.execute(f"sudo ip route list")
```

Setup routes on Node1

RTNETLINK answers: File exists

```
10.0.0.0/24 dev wg0 proto kernel scope link src 10.0.0.1
10.30.0.0/19 dev enp3s0 proto kernel scope link src 10.30.6.111 metric 100
10.30.6.11 dev enp3s0 proto dhcp scope link src 10.30.6.111 metric 100
169.254.169.254 via 10.30.6.11 dev enp3s0 proto dhcp src 10.30.6.111 metric 100
192.168.1.0/24 dev enp7s0 proto kernel scope link src 192.168.1.1
192.168.2.0/24 dev wg0 scope link
```

Setup routes on Node3

RTNETLINK answers: File exists

```
10.0.0.0/24 dev wg0 proto kernel scope link src 10.0.0.3
10.30.0.0/19 dev enp3s0 proto kernel scope link src 10.30.6.236 metric 100
10.30.6.11 dev enp3s0 proto dhcp scope link src 10.30.6.236 metric 100
169.254.169.254 via 10.30.6.11 dev enp3s0 proto dhcp src 10.30.6.236 metric 100
192.168.1.0/24 dev wg0 scope link
192.168.2.0/24 dev enp7s0 proto kernel scope link src 192.168.2.2
```

```
In [122... stdout, stderr = node1.execute(f'ping -c 5 {node3_net2_addr}')
```

```
PING 192.168.2.2 (192.168.2.2) 56(84) bytes of data.
64 bytes from 192.168.2.2: icmp_seq=1 ttl=63 time=0.443 ms
64 bytes from 192.168.2.2: icmp_seq=2 ttl=63 time=0.429 ms
64 bytes from 192.168.2.2: icmp_seq=3 ttl=63 time=0.343 ms
64 bytes from 192.168.2.2: icmp_seq=4 ttl=63 time=0.474 ms
64 bytes from 192.168.2.2: icmp_seq=5 ttl=63 time=0.380 ms
```

--- 192.168.2.2 ping statistics ---

```
5 packets transmitted, 5 received, 0% packet loss, time 4091ms
rtt min/avg/max/mdev = 0.343/0.413/0.474/0.046 ms
```

```
In [123... stdout, stderr = node1.execute(f'nc -zv {node3_net2_addr} 22')
```

Connection to 192.168.2.2 22 port [tcp/ssh] succeeded!

```
In [124... # =====
# Static Routing Verification
# =====
```

```
static_routes = ["192.168.2.0/24", "192.168.1.0/24"]
```

```

slice = fablib.get_slice(name="MySlice-tunnels-static-routes")
node1 = slice.get_node("Node1")
node2 = slice.get_node("Node2")
node3 = slice.get_node("Node3")

# 1. Bring down WireGuard to avoid interference
for node in [node1, node2, node3]:
    print(f"Bringing down WireGuard on {node.get_name()}...")
    node.execute("sudo wg-quick down wg0 || true") # ignore error if not running

# 2. Re-add static routes
print("\nRe-adding static routes...")
node1.execute("sudo ip route add 192.168.2.0/24 via 192.168.1.2 || true")
node3.execute("sudo ip route add 192.168.1.0/24 via 192.168.2.1 || true")

# 3. Show only static routes in routing tables
print("\n== Static Routes ==")
for node in [node1, node2, node3]:
    print(f"\n{node.get_name()} Routing Table:")
    out, _ = node.execute("ip route show")
    filtered = [line for line in out.splitlines() if any(r in line for r in static_
    if filtered:
        for line in filtered:
            print(line)
    else:
        print("(no static routes)")

# 4. Connectivity test: Node1 ↔ Node3 via static routing through Node2
print("\n== Static Routing Connectivity Test ==")

print("\nNode1 → Node3:")
out, _ = node1.execute("ping -c 3 192.168.2.2")
print(out)

print("\nNode3 → Node1:")
out, _ = node3.execute("ping -c 3 192.168.1.1")
print(out)

```

```

Bringing down WireGuard on Node1...
[#] ip link delete dev wg0
Bringing down WireGuard on Node2...
[#] ip link delete dev wg0
[#] iptables -D FORWARD -i wg0 -j ACCEPT; iptables -D FORWARD -o wg0 -j ACCEPT; iptables -t nat -D POSTROUTING -o enp7s0 -j MASQUERADE
Bringing down WireGuard on Node3...
[#] ip link delete dev wg0

```

Re-adding static routes...

== Static Routes ==

Node1 Routing Table:

```

10.30.0.0/19 dev enp3s0 proto kernel scope link src 10.30.6.111 metric 100
10.30.6.11 dev enp3s0 proto dhcp scope link src 10.30.6.111 metric 100
169.254.169.254 via 10.30.6.11 dev enp3s0 proto dhcp src 10.30.6.111 metric 100
192.168.1.0/24 dev enp7s0 proto kernel scope link src 192.168.1.1
192.168.2.0/24 via 192.168.1.2 dev enp7s0
192.168.1.0/24 dev enp7s0 proto kernel scope link src 192.168.1.1
192.168.2.0/24 via 192.168.1.2 dev enp7s0

```

Node2 Routing Table:

```

10.30.0.0/19 dev enp3s0 proto kernel scope link src 10.30.7.130 metric 100
10.30.6.11 dev enp3s0 proto dhcp scope link src 10.30.7.130 metric 100
169.254.169.254 via 10.30.6.11 dev enp3s0 proto dhcp src 10.30.7.130 metric 100
192.168.1.0/24 dev enp8s0 proto kernel scope link src 192.168.1.2
192.168.2.0/24 dev enp7s0 proto kernel scope link src 192.168.2.1
192.168.1.0/24 dev enp8s0 proto kernel scope link src 192.168.1.2
192.168.2.0/24 dev enp7s0 proto kernel scope link src 192.168.2.1

```

Node3 Routing Table:

```

10.30.0.0/19 dev enp3s0 proto kernel scope link src 10.30.6.236 metric 100
10.30.6.11 dev enp3s0 proto dhcp scope link src 10.30.6.236 metric 100
169.254.169.254 via 10.30.6.11 dev enp3s0 proto dhcp src 10.30.6.236 metric 100
192.168.1.0/24 via 192.168.2.1 dev enp7s0
192.168.2.0/24 dev enp7s0 proto kernel scope link src 192.168.2.2
192.168.1.0/24 via 192.168.2.1 dev enp7s0
192.168.2.0/24 dev enp7s0 proto kernel scope link src 192.168.2.2

```

== Static Routing Connectivity Test ==

Node1 → Node3:

```

PING 192.168.2.2 (192.168.2.2) 56(84) bytes of data.
64 bytes from 192.168.2.2: icmp_seq=1 ttl=63 time=0.224 ms
64 bytes from 192.168.2.2: icmp_seq=2 ttl=63 time=0.216 ms
64 bytes from 192.168.2.2: icmp_seq=3 ttl=63 time=0.211 ms

```

--- 192.168.2.2 ping statistics ---

```

3 packets transmitted, 3 received, 0% packet loss, time 2046ms
rtt min/avg/max/mdev = 0.211/0.217/0.224/0.005 ms
PING 192.168.2.2 (192.168.2.2) 56(84) bytes of data.
64 bytes from 192.168.2.2: icmp_seq=1 ttl=63 time=0.224 ms
64 bytes from 192.168.2.2: icmp_seq=2 ttl=63 time=0.216 ms
64 bytes from 192.168.2.2: icmp_seq=3 ttl=63 time=0.211 ms

```

```
--- 192.168.2.2 ping statistics ---  
3 packets transmitted, 3 received, 0% packet loss, time 2046ms  
rtt min/avg/max/mdev = 0.211/0.217/0.224/0.005 ms
```

Node3 → Node1:

```
PING 192.168.1.1 (192.168.1.1) 56(84) bytes of data.  
64 bytes from 192.168.1.1: icmp_seq=1 ttl=63 time=0.170 ms  
64 bytes from 192.168.1.1: icmp_seq=2 ttl=63 time=0.204 ms  
64 bytes from 192.168.1.1: icmp_seq=3 ttl=63 time=0.189 ms
```

```
--- 192.168.1.1 ping statistics ---  
3 packets transmitted, 3 received, 0% packet loss, time 2049ms  
rtt min/avg/max/mdev = 0.170/0.187/0.204/0.013 ms  
PING 192.168.1.1 (192.168.1.1) 56(84) bytes of data.  
64 bytes from 192.168.1.1: icmp_seq=1 ttl=63 time=0.170 ms  
64 bytes from 192.168.1.1: icmp_seq=2 ttl=63 time=0.204 ms  
64 bytes from 192.168.1.1: icmp_seq=3 ttl=63 time=0.189 ms
```

```
--- 192.168.1.1 ping statistics ---  
3 packets transmitted, 3 received, 0% packet loss, time 2049ms  
rtt min/avg/max/mdev = 0.170/0.187/0.204/0.013 ms
```

Delete the Slice

Please delete your slice when you are done with your experiment.

```
In [88]: slice.delete()
```

```
In [ ]:
```